

Head Soccer



Joaquín Avilés de la Fuente
Bruno

23 giugno 2026

Contents

1	Introduction & Environment Description	3
1.1	The Main Loop	3
2	External Libraries and Assets	5
2.1	gl-matrix.js	5
2.2	obj-loader.js	5
2.3	3D Complex Model and Textures	5
3	Technical Aspects	7
3.1	Hierarchical Models	7
3.2	Programmatic Animations	7
3.2.1	Walking Cycle and Head Bobbing	8
3.3	Lighting and Shaders (Blinn-Phong)	8
3.4	Cameras and View Matrices	8
3.5	Physics and Collisions	9
3.5.1	Kinematics	9
3.5.2	Collision Detection & Response	9
4	User Interaction	11
4.1	Gameplay Controls	11
4.2	Environment Interaction	11

1. Introduction & Environment Description

The development of the **3D Head Soccer** project was carried out using a pure graphics programming approach. Instead of relying on high-level 3D engines such as Three.js or Babylon.js, the environment is built directly upon **basic WebGL**. This decision was made to ensure a deep understanding of the graphics pipeline, matrix transformations, and shader programming.

Definición 1.1. WebGL (Web Graphics Library) is a JavaScript API for rendering interactive 2D and 3D graphics within any compatible web browser without the use of plug-ins. It is fully integrated with other web standards, allowing GPU-accelerated usage of physics and image processing.

The game logic, physics loop, and rendering pipeline are implemented entirely in vanilla **JavaScript (ES6)**. The HTML5 `<canvas>` element is used as the rendering surface. We utilize custom vertex and fragment shaders written in **GLSL (OpenGL Shading Language)** to compute lighting, materials, and transformations at the GPU level.

1.1 The Main Loop

The core of our application relies on the `requestAnimationFrame` API, which synchronizes our game loop with the browser's display refresh rate. In our `main.js` file, the `render(currentTime)` function is called recursively. It calculates the `deltaTime` to ensure smooth, frame-rate independent animations and physics updates, updates the `GameState`, updates the camera, and finally draws the scene.

2. External Libraries and Assets

In accordance with the project requirements, we developed the majority of the mathematical and physical logic from scratch. However, to handle complex linear algebra and 3D model parsing, we integrated specific external tools. All third-party libraries and assets not developed by the team are explicitly listed below:

2.1 gl-matrix.js

WebGL does not provide built-in matrix or vector mathematical classes. To perform transformations (translation, rotation, scaling) and to handle the View and Projection matrices, we included **gl-matrix.js**, a high-performance matrix and vector mathematics library for WebGL. We specifically rely on the `mat4`, `mat3`, and `vec3` structures.

2.2 obj-loader.js

To fulfill the requirement of including at least one complex model, we adapted a custom script to parse `.obj` 3D model files. Since raw `.obj` files do not always provide vertex normals (which are crucial for lighting), our loader calculates them dynamically using the cross product of the triangle edges.

```
1 // Calculate the normal of the triangle's surface (using cross product)
2 const a = [v2[0] - v1[0], v2[1] - v1[1], v2[2] - v1[2]];
3 const b = [v3[0] - v1[0], v3[1] - v1[1], v3[2] - v1[2]];
4 const normal = [
5   a[1] * b[2] - a[2] * b[1],
6   a[2] * b[0] - a[0] * b[2],
7   a[0] * b[1] - a[1] * b[0]
8 ];
```

Code 2.1: Procedural Walking Animation Math

2.3 3D Complex Model and Textures

- **Complex Model:** [Specify the model used, e.g., "Stadium bleachers" or "Trophy"], downloaded from [Source, e.g., Sketchfab]. This model is loaded asynchronously during the initialization phase.
- **Textures:** We programmatically generate procedural textures using the HTML5 Canvas 2D API (e.g., `createFieldTexture` for the grass and lines, and `createBallTexture` for the soccer ball pattern) to map them onto our WebGL geometries.

3. Technical Aspects

3.1 Hierarchical Models

A fundamental requirement of this project was the implementation of hierarchical models. In 3D graphics, a hierarchical model (or Scene Graph) links objects in a parent-child relationship. When a transformation is applied to the parent, it automatically affects all its children.

We achieved this by sequentially multiplying transformation matrices. Instead of using a traditional stack (`pushMatrix` / `popMatrix`), we utilized `mat4.clone()` to branch our transformation states.

For our **Head Soccer Players**, the body acts as the root node. The legs, arms, and head are children of this root. If the player translates across the field, the entire body structure translates cohesively.

```
// 1. Clone the root matrix (Body position)
const legBase = mat4.clone(root);
// 2. Translate to the hip joint relative to the body
mat4.translate(legBase, legBase, hip);
// 3. Apply rotation for the walking animation
mat4.rotateY(legBase, legBase, angle);
// 4. Render Upper Leg
const upperLeg = mat4.clone(legBase);
mat4.translate(upperLeg, upperLeg, [0, 0, -0.15]);
mat4.scale(upperLeg, upperLeg, [0.12, 0.12, 0.30]);
renderSolidObject(upperLeg, buffers.cube, pantsColor, 0.2);
// 5. Render Foot as a child of the leg
const foot = mat4.clone(legBase);
mat4.translate(foot, foot, [direction * 0.07, 0, -0.36]);
mat4.scale(foot, foot, [0.32, 0.16, 0.11]);
renderSolidObject(foot, buffers.cube, shoeColor, 0.32);
```

```
1 // 1. Clone the root matrix (Body position) const legBase = mat4.clone(root); // 2. Translate to the hip
  joint relative to the body mat4.translate(legBase, legBase, hip); // 3. Apply rotation for the walking
  animation mat4.rotateY(legBase, legBase, angle); // 4. Render Upper Leg const upperLeg =
  mat4.clone(legBase); mat4.translate(upperLeg, upperLeg, [0, 0, -0.15]); mat4.scale(upperLeg, upperLeg,
  [0.12, 0.12, 0.30]); renderSolidObject(upperLeg, buffers.cube, pantsColor, 0.2); // 5. Render Foot as a
  child of the leg const foot = mat4.clone(legBase); mat4.translate(foot, foot, [direction * 0.07, 0,
  -0.36]); mat4.scale(foot, foot, [0.32, 0.16, 0.11]); renderSolidObject(foot, buffers.cube, shoeColor,
  0.32);
```

Code 3.1: Procedural Walking Animation Math

This approach ensures that the foot stays attached to the bottom of the leg, regardless of how much the leg rotates during the walking or kicking animations.

3.2 Programmatic Animations

As per the project constraints, animations could not be imported from external files (e.g., skeletal animations via mixamo). All animations were implemented programmatically in JavaScript.

3.2.1 Walking Cycle and Head Bobbing

The walking animation relies on trigonometric functions parameterized by a continuous time variable (`walkPhase`), which increments based on `deltaTime` and player input.

```

1 // In renderPlayer()
2 const walking = Math.sin(player.walkPhase) * 0.55;
3 const kick = player.kickTimer > 0 ? -player.direction * 1.05 * Math.sin((player.kickTimer /
    0.22) * Math.PI) : 0;
4 const headBob = player.onGround ? Math.abs(Math.sin(player.walkPhase)) * 0.025 : 0.045;
5
6 const leftAngle = walking + (player.direction < 0 ? kick : 0);
7 const rightAngle = -walking + (player.direction > 0 ? kick : 0);

```

Code 3.2: Procedural Walking Animation Math

This sine-wave approach creates a natural oscillating pendulum movement for the arms and legs, while the absolute value `Math.abs(Math.sin(...))` causes the head to bob upwards twice per step, mimicking a real running cycle.

3.3 Lighting and Shaders (Blinn-Phong)

Lighting is calculated per-pixel in our custom GLSL fragment shaders. We implemented a **Blinn-Phong reflection model**, which combines three lighting components:

1. **Ambient:** Base illumination.
2. **Diffuse (Lambertian):** Light scattered in all directions based on the dot product of the surface normal and the light direction.
3. **Specular:** The shiny highlight, calculated using the halfway vector (or reflection vector) and the view direction.

We use an array of three point lights (`u_lightPositions[3]`), representing two stadium lights and a dynamic light attached to the ball. We also apply a realistic distance attenuation formula:

```

1 float distance = length(u_lightPositions[i] - v_worldPosition);
2 float attenuation = 1.0 / (1.0 + 0.08 * distance + 0.028 * distance *
    distance);
3 float diff = max(dot(normal, lightDir), 0.0);
4 vec3 reflectDir = reflect(-lightDir, normal);
5 float spec = pow(max(dot(viewDir, reflectDir), 0.0), 52.0) *
    u_specularStrength;
6 color += (u_color * diff + vec3(spec)) * attenuation * u_lightIntensity;

```

Code 3.3: Fragment Shader Lighting Calculation

3.4 Cameras and View Matrices

The `Camera` class constructs the View and Projection matrices. The Projection matrix uses `mat4.perspective` to create a realistic 3D frustum with a Field of View (FOV) of 45 degrees ($\pi/4$).

We implemented three interactive camera modes:

- **Side View:** A static broadcast-like view from the stands.
- **Top View:** An overhead tactical view.
- **Stadium View:** A dynamic camera that actively interpolates its target and position based on the ball's X-axis coordinate, keeping the action centered.

3.5 Physics and Collisions

To maintain performance and exact control over the arcade feel of "Head Soccer", we built a custom physics engine.

3.5.1 Kinematics

We use Euler integration to update positions based on velocities and gravity:

```
1 const gravity = -7.8;
2 this.velocity.z += gravity * deltaTime;
3 this.position.x += this.velocity.x * deltaTime;
4 this.position.z += this.velocity.z * deltaTime;
```

Code 3.4: Procedural Walking Animation Math

3.5.2 Collision Detection & Response

Player-ball collisions are approximated using 2D circles (projected onto the XZ plane) for different body parts (head, body, foot). We compute the Euclidean distance between the ball's center and the body part's center. If the distance is less than the sum of their radii, a collision occurs.

We resolve the collision by pushing the ball outside the player along the normal vector, and transferring momentum (velocity) from the player to the ball, including extra "kick power" if the player is currently executing a kick action.

4. User Interaction

The game features dynamic user interaction via keyboard inputs, satisfying the requirement to allow users to manipulate both the gameplay and the rendering environment in real-time.

4.1 Gameplay Controls

We capture key presses using a dictionary mapped to `keydown` and `keyup` events, preventing the default browser scrolling.

- **Player 1:** Moves horizontally using A and D, jumps with W, and kicks with S.
- **Player 2:** Moves using Arrow Left/Right, jumps with Arrow Up, and kicks with Arrow Down.

4.2 Environment Interaction

Depending on the user's preferences, the visual and logical environment can be modified:

- **Cameras (Keys 1, 2, 3):** Switch between Side, Top, and dynamic Stadium views by reconfiguring the `camera.position` and `camera.target` vectors used in `mat4.lookAt`.
- **Difficulty (Keys + and -):** Adjusts the physical parameters of the `GameState`, such as the maximum speed of the ball, player movement multipliers, and shot power multipliers.
- **Stadium Lights (Key O):** Toggles the boolean `stadiumLightsOn`. This alters the `u_lightIntensity` uniform passed to the GLSL shaders (switching between 1.0 and 0.22), simulating turning off the main stadium reflectors for a night-time atmosphere.
- **Help Menu (Key H):** Toggles the display of the HTML/CSS overlay containing the control instructions.